

Fundamentos de redes neuronales artificiales

Redes convolucionales

Emiliano Damián Churruca

Laboratorio de Investigación y Desarrollo en
Computación(LIDEC)

Universidad Nacional de Hurlingham

(Material desarrollado por Juan Miguel Santos y adaptado
para este curso)

Redes neuronales artificiales

Convolución

- La convolución (correlación) es la integral del producto entre dos funciones.
- Sea $g(x)$ y $f(x)$ dos funciones $f, g : \mathbb{R} \rightarrow \mathbb{R}$, definimos la convolución entre f y g como

$f * g : \mathbb{R} \rightarrow \mathbb{R}$ tal que

$$f * g(t) = \int_{-\infty}^{+\infty} f(\tau)g(t + \tau)d\tau$$

Convolución vs Correlación (Ejemplo numérico)

Señal y filtro

- Señal $f = [1, 2, 3]$
- Filtro $g = [0, 1, 0,5]$

Correlación cruzada

(Sin invertir el filtro)

$$f \star g[0] = 1 \cdot 1 + 2 \cdot 0,5 = 2$$

$$f \star g[1] = 2 \cdot 1 + 3 \cdot 0,5 = 3,5$$

$$f \star g[2] = 3 \cdot 1 + 0 \cdot 0,5 = 3$$

$$\Rightarrow f \star g = [2, 3,5, 3]$$

Convolución

(Invirtiendo el filtro)

$$g_{\text{inv}} = [0,5, 1, 0]$$

$$f * g[0] = 1 \cdot 1 = 1$$

$$f * g[1] = 1 \cdot 0,5 + 2 \cdot 1 = 2,5$$

$$f * g[2] = 2 \cdot 0,5 + 3 \cdot 1 = 4$$

$$\Rightarrow f * g = [1, 2,5, 4]$$

Redes neuronales artificiales

Convolución

- ✧ Esta definición se puede extender a funciones f y g definidas en $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$:

$$f * g(t_x, t_y) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f(\tau_x, \tau_y) g(t_x + \tau_x, t_y + \tau_y) d\tau_x d\tau_y$$

Redes neuronales artificiales

Convolución

- ✚ Ver video de convolución de dos funciones constantes con bases acotadas
(extraído de [https://es.m.wikipedia.org/wiki/](https://es.m.wikipedia.org/wiki/Archivo:Convolucion_Funcion_Pi.gif) Archivo:Convolucion_Funcion_Pi.gif)

Redes neuronales artificiales

Convolución

- ✚ Si f y g son funciones discretas podemos expresar la convolución entre dichas funciones como:

$$f * g[t] = \sum_{k=a}^b f[k]g[t + k]$$

Redes neuronales artificiales

Convolución

- ✚ La utilidad de la convolución depende de dónde se esté aplicando.
- ✚ Por ejemplo, se puede usar para suavizar una imagen.
- ✚ O se puede usar para filtrar una señal.
- ✚ O para saber cómo es la respuesta de un dispositivo electrónico (capacitor) frente a un impulso eléctrico.
- ✚ Etcétera.

Redes neuronales artificiales

Convolución

- ✧ Y en el caso que f dependa de x y de y (como por ejemplo, si f depende la intensidad de cada pixel en una imagen),
- ✧ y g esté definido como una grilla de 3×3 , la convolución de g sobre f (ambas discretas) está definida como:

$$f * g[x, y] = \sum_{k_x=-1}^1 \sum_{k_y=-1}^1 f[k_x, k_y] g[x + k_x, y + k_y]$$

Redes neuronales artificiales

Convolución

- La función f es la imagen y la función g es el filtro o **núcleo** de la convolución.
- Si definimos un núcleo como:

$$g_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Redes neuronales artificiales

Convolución

✚ y otro núcleo como:

$$g_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Redes neuronales artificiales

Convolución en imágenes

- ✧ Se puede calcular una estimación del gradiente de la intensidad de una imagen en cada pixel de la misma, sea vertical (g_x) u horizontal (g_y).
- ✧ Dicho filtro es muy usado en imágenes y se denomina filtro de Sobel u Operador de Sobel.
- ✧ Se utiliza para la detección de bordes.
- ✧ Si el gradiente es muy alto, es probable que haya un borde.

Redes neuronales artificiales

Convolución en imágenes

Aplicación de un filtro para detección de bordes verticales

Original



Sobel X



Redes neuronales artificiales

Convolución en imágenes

Aplicación de un filtro para detección de bordes horizontales



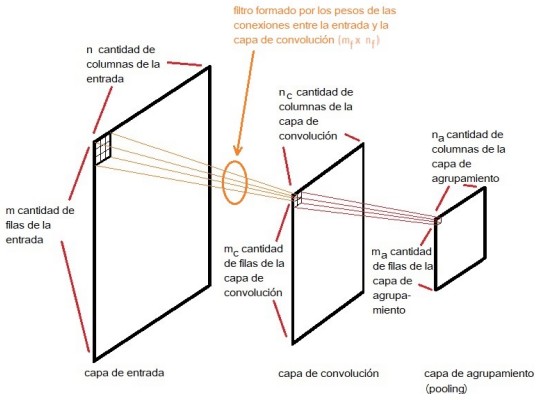
Redes neuronales artificiales

Estructura básica de una capa de convolución

- Sea la **capa de entrada** un arreglo de m filas por n columnas.
- La **capa de convolución** será un arreglo de m_c filas $\times$ n_c columnas de neuronas.
- La **capa de agrupamiento** será un arreglo de m_a filas $\times$ n_a columnas de neuronas.
- El **filtro** será un arreglo de $m_f \times n_f$ pesos que conectan de a grupos de $m_f \times n_f$ entradas con cada una de las neuronas de la capa de convolución.

Redes neuronales artificiales

Estructura de una red neuronal artificial convolucional



Redes neuronales artificiales

Estructura básica de una capa de convolución

- Llamemos $X_{i,j}$ al valor de la entrada (i,j) donde $0 \leq i < m$ y $0 \leq j < n$
- Llamemos V_{i_c,j_c} al estado de activación de cada neurona (i_c,j_c) de la capa de convolución donde $0 \leq i_c < m_c$ y $0 \leq j_c < n_c$.
- Llamemos w al filtro cuyas dimensiones son $m_f \times n_f$ (m_f filas y n_f columnas).
- Luego, w_{k_i,k_j} será cada uno de los componentes de la matriz del filtro con $0 \leq k_i < m_f$ y $0 \leq k_j < n_f$.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Para calcular el estado de activación de la neurona (i, j) de la capa de convolución seguimos

$$V_{i,j} = g(\sum_{k_i=0}^{m_f-1} \sum_{k_j=0}^{n_f-1} w_{k_i,k_j} X_{i+k_i,j+k_j} + u)$$

donde u es el valor de umbral para el filtro y $g()$ es la función de activación (por ejemplo, $\tanh()$ o ReLU).

- Es decir, para el cálculo del estado de activación de cualquier neurona de la capa convolucional se usa el mismo filtro w .

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ El tamaño de m_c y n_c dependen de dos factores.
- ✚ Del **salto** (o *stride*) que indica cada cuanto se desliza el filtro sobre la entrada (por defecto *salto* = 1).
- ✚ Del tamaño del filtro.

Si por ejemplo, la entrada es de 10×10 y el filtro es de 3×3 , entonces el tamaño de la capa de convolución será de 8×8 pues $10 - 3 + 1 = 8$.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Para evitar este problema, se suele considerar una entrada con un tamaño mayor donde se le agregan filas y columnas para que el tamaño de la capa de convolución sea el mismo que originalmente tenía la entrada.
- Este procedimiento se llama **relleno** (o *padding*).

Redes neuronales artificiales

Estructura básica de una capa de convolución

- En nuestro ejemplo, bastará con agregarle 1 fila antes de la primera fila y una después de la última.
Análogamente se procede con las columnas.
- Así, en el ejemplo que vimos recién, la capa de convolución quedará de 10×10 ya que
$$10 + 2 - 3 + 1 = 10$$
donde el '+2' surge del relleno que agregamos.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Un filtro detectará una característica particular (*feature*).
- La característica detectada dependerá del aprendizaje, es decir el valor de los coeficientes aprendido durante el entrenamiento.
- Sin embargo, si se desea caracterizar bien una entrada puede ser necesario aprender a detectar más de una característica.
- A tal fin, una red neuronal artificial del tipo convolucional (o simplemente, red convolucional) puede tener más de un filtro.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ Por cada filtro que se agrega en una red convolucional, se agrega una capa convolucional.
- ✚ Es decir, si tenemos F filtros entonces tendremos F capas convolucionales.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ Por lo tanto, para obtener el estado de activación de una neurona de la capa convolucional 0 se debe usar el conjunto de pesos w del filtro 0, para la capa convolucional 1 se debe usar el w del filtro 1, y así para cada capa convolucional.
- ✚ Recordar que cada filtro tiene su umbral, de modo que si tenemos F filtros, deberemos usar F umbrales diferentes.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ ¿Qué ocurre si nuestra capa de entrada no es solo una matriz sino un tensor?
- ✚ Esto ocurre, por ejemplo, cuando la entrada son imágenes color que usualmente tienen 3 canales (RGB o BGR, entre otras posibilidades).
- ✚ En este caso, si por ejemplo el filtro para una imagen de un solo canal es de 3×3 , ahora será de $3 \times 3 \times 3$, es decir, un tensor de 3 dimensiones.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Para calcular el estado de activación de la neurona (i, j) de la capa de convolución correspondiente al filtro f seguimos

$$V_{f,i,j} = g\left(\sum_{c=0}^{canales-1} \sum_{k_i=0}^{m_f-1} \sum_{k_j=0}^{n_f-1} w_{c,k_i,k_j} X_{c,i,j} + u_f\right)$$

donde c es el índice para los canales de la capa de entrada,

canales es la cantidad de canales, y

u_f es el umbral para el filtro f .

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Una vez obtenidos los estados de activación de las neuronas de la capa de convolución la información contenida en ellas suele ser 'agrupada' de a parches de dimensiones $m_a \times n_a$ donde lo usual es que $m_a = n_a$, es decir, parches cuadrados.
- Este tipo de procesamiento lo realiza la capa de agrupamiento (o *pooling*).
- Hay varias formas de realizar esto pero lo más usual es tomar el máximo de un parche de la capa de convolución.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- De forma similar a como se desliza el filtro sobre la capa de entrada, se desliza una ventana sobre la capa de convolución para obtener el valor de activación de las neuronas de la capa de agrupamiento:

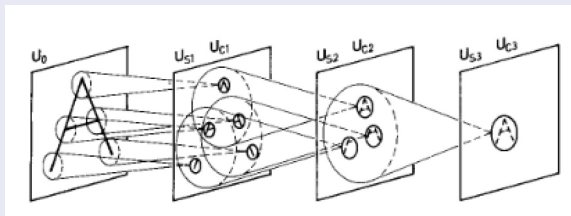
$$V_{f,i_a,j_a}^a = F_a(V_{f,i,j}, \forall (i,j) \text{ en } \textit{Parche})$$

donde F_a es la función de agrupamiento,
 $\textit{Parche}$ es el conjunto de pares que satisfacen
 $(i_a * \textit{salto}_a + i, j_a * \textit{salto}_a + j)$ con
 $0 \leq i < m_a, 0 \leq j < n_a$, y
 $\textit{salto}_a$ (o *pooling stride*) es el salto que realiza la
ventana cuando se mueve entre parche y parche.

Redes neuronales artificiales

Estructura básica de una capa de convolución

La intención es agrupar características que se hayan detectado en la capa de convolución.



(extraído de Kunihiro Fukushima and Nobuaki Wake. Handwritten alphanumeric character recognition by the neocognitron. IEEE transactions on Neural Networks, 2(3):355-365, 1991.)

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ La idea es lograr la **invarianza traslacional**.
- ✚ Esto es, que el desempeño de la red (como clasificador) no dependa de la ubicación de las características detectadas por medio de los filtros de la capa convolucional.
- ✚ Claramente, esto dependerá del tamaño del parche y además del número de capas de agrupamiento en la red.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- ✚ La función de agrupamiento $F_a()$ usualmente es definida como:

$$F_a(A) = \max(A)$$

donde A es el conjunto de valores de activación de las neuronas en la capa de convolución correspondientes a un parche.

- ✚ Como alternativas a $\max()$ se pueden considerar el promedio, la mediana o cualquier otra que cumpla un propósito de diseño.

Redes neuronales artificiales

Estructura básica de una capa de convolución

- Las conexiones entre la capa de convolución y la de agrupamiento son fijas, es decir, no serán afectados sus pesos durante el entrenamiento de la red.
- Sin embargo, a través de las mismas sí se retropropaga el error desde las salidas hacia la entrada.

Redes neuronales artificiales

Estructura completa de una red convolucional

- ✧ El esquema de una capa de convolución + una capa de agrupamiento puede repetirse varias veces a largo de la red.
- ✧ Luego de la última capa de agrupamiento, lo usual es que haya una o más capas completamente conexas (las típicas de un perceptron multicapa).

Redes neuronales artificiales

Estructura completa de una red convolucional

- Cada una de las neuronas de la capa de agrupamiento estará conectada con todas las neuronas de la primera capa completamente conexa, aún cuando las neuronas de la capa de agrupamiento estén representadas por un tensor de 3 dimensiones (situación usual si se trabaja con más de un filtro).

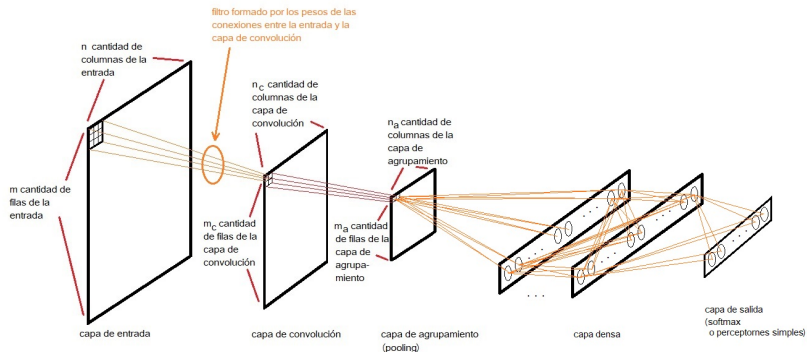
Redes neuronales artificiales

Estructura completa de una red convolucional

- Una forma de pensar esta situación es cambiando la forma de tensor de la capa oculta (*reshape* en inglés) de 3 dimensiones a una dimensión
$$m \times n \times p \rightarrow 1 \times m * n * p,$$
- y luego tratarla como una entrada de $m * n * p$ unidades.

Redes neuronales artificiales

Estructura completa de una red convolucional



Redes neuronales artificiales

Estructura completa de una red convolucional

- Finalmente, la última capa es la de salida.
- Si el objetivo de la red convolucional es ajustar, para cada entrada, uno o más valores de salida, la última capa de la red convolucional corresponderá a la última capa completamente conexa.
- Si el objetivo es clasificar, la última capa será una **softmax**.

Redes neuronales artificiales

Estructura completa de una red convolucional

- La función de activación de las unidades de la capa softmax viene dada por:

$$g_i(h_1, h_2, \dots, h_{n_{sm}}) = \frac{e^{h_i}}{\sum_{k=1}^{n_{sm}} e^{h_k}}$$

donde h_i es el estado de excitación de la neurona i de la capa softmax (incluyendo su umbral),

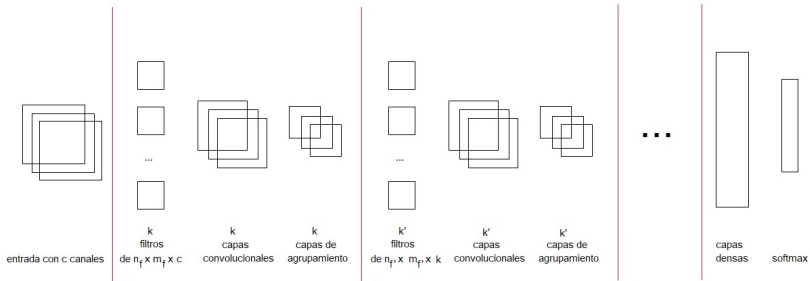
n_{sm} es la cantidad de unidades de la capa softmax,

$g_i()$ es la función de activación de la unidad i , y

$$1 \leq i \leq n_{sm}.$$

Redes neuronales artificiales

Esquema general de una red convolucional



Redes neuronales artificiales

Inicio repaso de retropropagación en PMC.

Redes neuronales artificiales en PMC

Repaso de retropropagación

- Sea la siguiente función de error $E()$:

$$E(w) = \frac{1}{2} \sum_{k=1}^{num_{sal}} (Y_k - V_k^L)^2$$

donde num_{sal} es la cantidad de unidades de la capa de salida, Y_k es la salida deseada para la unidad k de dicha capa, V_k^L es el estado de activación para la unidad k también de dicha capa, y L la cantidad de capas de la red.

- Sea w_{ij}^l el peso de la conexión entre la unidad j de la capa $l - 1$ y la unidad i de la capa l .

Redes neuronales artificiales

Repaso de retropropagación en PMC

- La corrección del peso w_{ij}^l es:

$$\Delta w_{ij}^l = -\eta \frac{\partial E}{\partial w_{ij}^l}$$

- La $\frac{\partial E}{\partial w_{ij}^l}$ la podemos obtener según:

$$\frac{\partial E}{\partial w_{ij}^l} = \frac{\partial E}{\partial h_i^l} \frac{\partial h_i^l}{\partial w_{ij}^l}.$$

- Dado que h_i^l es

$$h_i^l = \sum_k w_{ik}^l * V_k^{l-1}$$

entonces

$$\frac{\partial h_i^l}{\partial w_{ij}^l} = V_j^{l-1}.$$

Redes neuronales artificiales

Repaso de retropropagación en PMC

✚ Por otro lado, $\frac{\partial E}{\partial h_i^l}$ lo podemos escribir como:

$$\begin{aligned}\frac{\partial E}{\partial h_i^l} &= \frac{\partial E}{\partial V_i^l} \frac{\partial V_i^l}{\partial h_i^l} = \\ &= \frac{\partial E}{\partial V_i^l} g'(h_i^l).\end{aligned}$$

✚ Si $l = L$ (L es la cantidad de capas de la red):

$$\frac{\partial E}{\partial V_i^L} = -(Y_i - V_i^L)$$

y por lo tanto

$$\frac{\partial E}{\partial h_i^l} = -(Y_i - V_i^L)g'(h_i^l)$$

Redes neuronales artificiales

Repaso de retropropagación en PMC

✚ Si $l \neq L$ entonces

$$\frac{\partial E}{\partial V_i^l} = \sum_k \frac{\partial E}{\partial h_k^{l+1}} \frac{\partial h_k^{l+1}}{\partial V_i^l} = \sum_k \frac{\partial E}{\partial h_k^{l+1}} w_{ki}^{l+1}$$

dado que

$$h_k^{l+1} = \sum_{i'} w_{ki'}^{l+1} * V_{i'}^l.$$

✚ Observar que $\frac{\partial E}{\partial h_k^{l+1}}$ para la capa $l + 1$ ya fue calculado previamente.

Redes neuronales artificiales

Repaso de retropropagación en PMC. Importante.

- ✚ En la salida, el error sobre cada unidad lo podemos expresar como:

$$\frac{\partial E}{\partial V_i^L} = -(Y_i - V_i^L)$$

y el δ que se retropropaga es

$$\frac{\partial E}{\partial h_i^L} = -(Y_i - V_i^L)g'(h_i^L).$$

- ✚ Para la capa oculta l , el error sobre cada unidad es:

$$\frac{\partial E}{\partial V_i^l} = \sum_k \frac{\partial E}{\partial h_k^{l+1}} w_{ki}^{l+1}$$

y el δ que se retropropaga es:

$$\frac{\partial E}{\partial h_i^l} = \frac{\partial E}{\partial V_i^l} g'(h_i^l).$$

Redes neuronales artificiales

Fin repaso.

Redes neuronales artificiales

Retropropagación en la capa de convolución

- Para actualizar los pesos de los filtros debemos calcular $\frac{\partial E}{\partial w_{i_f j_f}}$ según:

$$\begin{aligned}\frac{\partial E}{\partial w_{i_f j_f}} &= \sum_{i=0}^{m_c-1} \sum_{j=0}^{n_c-1} \frac{\partial E}{\partial h_{ij}^c} \frac{\partial h_{ij}^c}{\partial w_{i_f j_f}} = \\ &= \sum_{i=0}^{m_c-1} \sum_{j=0}^{n_c-1} \frac{\partial E}{\partial h_{ij}^c} V_{i+i_f j+j_f}\end{aligned}$$

donde $0 \leq i_f < m_f$, $0 \leq j_f < n_f$,

$V_{i+i_f j+j_f}$ es el estado de activación de las unidades de la entrada o de una capa de agrupamiento previa, y se asume que la capa previa tiene relleno (*padding*).

- h_{ij}^c es el estado de excitación de la neurona ij de la capa de convolución.

Redes neuronales artificiales

Retropropagación en la capa de convolución

✚ $\frac{\partial E}{\partial h_{ij}^c}$ es el δ retropropagado de capas superiores.

✚ Para calcular los $\delta \frac{\partial E}{\partial h_{ij}^c}$ seguir:

$$\frac{\partial E}{\partial h_{ij}^c} = \frac{\partial E}{\partial V_{ij}^c} \frac{\partial V_{ij}^c}{\partial h_{ij}^c} = \frac{\partial E}{\partial V_{ij}^c} g'(h_{ij}^c)$$

donde V_{ij}^c y h_{ij}^c son el estado de activación y de excitación de la unidad ij de la capa convolucional.

Redes neuronales artificiales

Retropropagación en la capa de convolución

✚ El error retropropagado sobre la convolucional es

$$\frac{\partial E}{\partial V_{ij}^c} = \sum_k \frac{\partial E}{\partial h_k^{prox}} w_{k,ij}$$

donde h_k^{prox} es el estado de excitación de la neurona k de la próxima capa,

$w_{k,ij}$ es el peso de la conexión entre la neurona de la capa convolucional ij y la neurona k de la próxima capa, y

k es un índice (usualmente un par, o una terna en la capa de agrupamiento).

Redes neuronales artificiales

Retropropagación en la capa de convolución

- Para calcular los δ sobre la capa previa debemos primero calcular $\frac{\partial E}{\partial V_{ij}^{previa}}$ según

$$\begin{aligned}\frac{\partial E}{\partial V_{ij}^{previa}} &= \sum_{i_f=0}^{m_f-1} \sum_{j_f=0}^{n_f-1} \frac{\partial E}{\partial h_{i-i_f+1 \ j-j_f+1}^c} \frac{\partial h_{i-i_f+1 \ j-j_f+1}^c}{\partial V_{ij}^{previa}} = \\ &= \sum_{i_f=0}^{m_f-1} \sum_{j_f=0}^{n_f-1} \frac{\partial E}{\partial h_{i-i_f+1 \ j-j_f+1}^c} w_{i_f \ j_f}.\end{aligned}$$

- Fijarse que los índices de las posiciones del filtro se substraen de los índices de h ya que el δ para la unidad ij de la capa previa se 'nutre' a través de las conexiones del filtro cuyos índices tienen el orden invertido respecto a los de la capa convolucional.

Redes neuronales artificiales

Retropropagación en la capa de agrupamiento (*pooling*)

- ✚ La conexiones entre las unidades de la capa de convolución y la capa de agrupamiento no están sujetas a modificación y por lo tanto no se debe calcular $\frac{\partial E}{\partial w}$.

Redes neuronales artificiales

Retropropagación en la capa de agrupamiento (*pooling*)

- ✚ Para calcular cómo se retropropaga el error a través de esta capa consideraremos el caso más común que es cuando la función de activación en la capa de agrupamiento es $F_a(A) = \max(A)$ con A como definimos previamente.
- ✚ Para ese caso, el δ sobre la capa de agrupamiento se propaga directamente sobre la unidad de la capa de convolución que resultó ser de la máxima activación, mientras que para las restantes el δ retropropagado es 0.

Redes neuronales artificiales

Retropropagación en la capa softmax

- Para calcular los Δw de cada uno de los pesos de la capa softmax debemos calcular $\frac{\partial E}{\partial w_{ij}}$ como

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial h_i} \frac{\partial h_i}{\partial w_{ij}} = \frac{\partial E}{\partial h_i} V_j^{l-1}.$$

- Para calcular $\frac{\partial E}{\partial h_i}$ hay que tener en cuenta que E depende de todos los estados de activación de la capa de salida:

$$E = \frac{1}{2} \sum_k (Y_k - V_k)^2$$

y todos los V_k dependen de h_i .

- Por lo tanto, aplicando la regla de la derivada total, tenemos que:

$$\frac{\partial E}{\partial h_i} = \sum_k \frac{\partial E}{\partial V_k} \frac{\partial V_k}{\partial h_i}.$$

Redes neuronales artificiales

Retropropagación en la capa softmax

✚ La $\frac{\partial E}{\partial V_k}$ la podemos calcular según:

$$\frac{\partial E}{\partial V_k} = \frac{\partial \frac{1}{2} \sum_{k'} (Y_{k'} - V_{k'})^2}{\partial V_k} = -(Y_k - V_k).$$

✚ El cálculo de $\frac{\partial V_k}{\partial h_i}$ requiere un poco más de detalle.

Redes neuronales artificiales

Retropropagación en la capa softmax

✚ La $\frac{\partial V_k}{\partial h_i}$ es

$$\frac{\partial V_k}{\partial h_i} = \frac{\partial \frac{e^{h_k}}{\sum_{k'} e^{h_{k'}}}}{\partial h_i}.$$

✚ Por comodidad, definimos S como:

$$S = \sum_{k'} e^{h_{k'}}.$$

✚ Como lo que hay que derivar es un cociente tenemos:

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{\frac{\partial e^{h_k}}{\partial h_i} S - e^{h_k} \frac{\partial S}{\partial h_i}}{S^2}.$$

✚ La $\frac{\partial S}{\partial h_i} = e^{h_i}$.

Redes neuronales artificiales

Retropropagación en la capa softmax

✚ En el caso que $i = k$

$$\frac{\partial e^{h_k}}{\partial h_i} = e^{h_i}.$$

$$\text{Así, } \frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{e^{h_i} S - e^{h_i} e^{h_i}}{S^2}$$

o, lo que es lo mismo

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{e^{h_i} S - e^{h_k} e^{h_i}}{S^2}.$$

Operando

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{e^{h_i} S - e^{h_k} e^{h_i}}{S^2} = \frac{e^{h_i} (S - e^{h_k})}{SS} =$$

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = V_i(1 - V_k).$$

Redes neuronales artificiales

Retropropagación en la capa softmax

✚ En el caso que $i \neq k$

$$\frac{\partial e^{h_k}}{\partial h_i} = 0.$$

Así, $\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{-e^{h_k} e^{h_i}}{S^2}$

o, lo que es lo mismo

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \frac{-e^{h_k} e^{h_i}}{SS} = -V_k V_i.$$

✚ Resumiendo

$$\frac{\partial \frac{e^{h_k}}{S}}{\partial h_i} = \begin{cases} V_i(1 - V_j) & \text{si } i = k \\ -V_i V_j & \text{si } i \neq k \end{cases}.$$

Redes neuronales artificiales

Cómo crear una red convolucional con keras

```
model = Sequential()
model.add(Conv2D(input_shape=(32, 32, 3), kernel_size=(2, 2),
padding='same', strides=(2, 2), filters=32))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(1, 1),
padding='same'))
model.add(Conv2D(kernel_size=(2, 2),
padding='same', strides=(2, 2), filters=64))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(1, 1),
padding='same'))
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

Redes neuronales artificiales

Cómo crear una red convolucional con keras

- ✚ ¿Cómo usar el relleno (*padding*)?
- ✚ Hay 3 tipos de relleno en keras:
 - ✚ *valid*. No se rellena y la dimensión de la capa convolucional resulta de menor tamaño que la de entrada.
 - ✚ *same*. Se rellena con ceros para que la capa de convolución tenga las mismas dimensiones que la entrada.
 - ✚ *causal* Solo usada en capas del tipo Conv1D.

Redes neuronales artificiales

Cómo crear una red convolucional con keras

- ✚ ¿Cómo especificar el salto del filtro en la imagen (*stride*)?
- ✚ Se realiza mediante el argumento *strides* donde se especifica con un par cuya primer componente es el salto en las columnas de la imagen y el segundo en las filas

Redes neuronales artificiales

Cómo crear una red convolucional con keras

- ¿Para qué se usa la capa *Flatten*?
- Permite reformar una capa de varias dimensiones (por ejemplo, un tensor de 2 dimensiones o de 3 dimensiones) en una capa de 1 dimensión (como las típicas capas de un perceptron multicapa).